Decomposing Algebraic Numbers into Low-Degree Roots

Certified Mathematica searches, global degree bounds, and exact normal-field algorithms

Prepared for Vladimir Reshetnikov

September 6, 2026

Abstract

Given an exact algebraic number, one can seek a flat sum or product of algebraic numbers whose largest degree over $\mathbb{Q}$ is minimal. This is not polynomial factorization, and finding an identity is not the same as proving optimality. We solve both degree-nine examples in the question, with the globally minimal largest degree equal to three, even when arbitrarily many parts are permitted. A practical resultant algorithm recovers the second polynomial from a candidate first polynomial; a bounded search needs no advance knowledge of either cubic. We also prove finite, complete methods for unrestricted finite sums and for products of at most two factors, using normal fields, trace, norm, and rational linear algebra. For arbitrary-length products, we supply a bounded dictionary search and independent optimality certificates, but do not claim a complete global optimizer. The accompanying Wolfram Language package, examples, and tests make these distinctions explicit. Independent exact symbolic checks are provided; the Wolfram test suite has not been run in a Wolfram kernel in this environment.

What is completely solved here?

The two examples are solved *globally*, not merely within an ansatz. The unrestricted additive problem has a finite exact algorithm. The unrestricted *two-factor* multiplicative problem also has a finite exact algorithm. The arbitrary-number-of-factors multiplicative problem receives a useful certifying search, not an unconditional algorithm for its optimum. A resource limit, a failed coefficient search, and a mathematical impossibility certificate are different outcomes throughout.

Contents

1 The problem and the answers to the two examples

Let

$$f(x) = x^3 + x + 1, \qquad g(x) = x^3 - x + 1, \tag{1}$$

$$p_\times(x) = x^9 + 2x^7 - 3x^6 + x^5 - x^4 + 3x^3 - x - 1, \tag{2}$$

$$p_+(x) = x^9 + 6x^6 + 3x^5 - 15x^3 + 24x^2 - 4x + 8. \tag{3}$$

Write a and b for the unique real roots of f and g. The requested identities are

$$\boxed{\alpha_\times = ab, \qquad \alpha_+ = a + b.} \tag{4}$$

In Mathematica notation, the complete input is:

```
a = Root[1 + # + #^3 &, 1];
b = Root[1 - # + #^3 &, 1];
productTarget = Root[
  -1 - # + 3 #^3 - #^4 + #^5 - 3 #^6 + 2 #^7 + #^9 &, 1];
sumTarget = Root[
  8 - 4 # + 24 #^2 - 15 #^3 + 3 #^5 + 6 #^6 + #^9 &, 1];

RootReduce[productTarget - a b]
RootReduce[sumTarget - a - b]
(* Both expressions are exactly 0. *)
```

The assertions in this listing follow from the exact polynomial identities and branch arguments below; they are not presented as a transcript of a Wolfram-kernel run. `RootReduce` is the appropriate exact normalization operation, rather than a numerical comparison [3].

The numerical values, for orientation only, are

$$\begin{aligned} a &= -0.682327803828019327369483739711\ldots, \\ b &= -1.324717957244746025960908854478\ldots, \\ ab &= 0.903891894458347561098959026166\ldots, \\ a + b &= -2.007045761072765353330392594189\ldots. \end{aligned}$$

The two cubics and both degree-nine polynomials are irreducible over $\mathbb{Q}$ and have exactly one real root. These facts were independently checked by exact polynomial arithmetic and real-root counting. Consequently the resultant identities identify the actual real roots, not just unspecified conjugates. Mathematica numbers real roots first, in increasing order, so index 1 selects the required root in every case [2].

Most importantly, the maximum degree three is *minimal over all finite sums or products*. A number of degree nine cannot belong to a field generated by finitely many quadratic numbers: such a field is contained in a finite multiquadratic extension, whose degree is a power of two. Section 5 proves a more general version of this obstruction.

1.1 Running the supplied search

Extract the archive and load the package from its directory:

```
Get["RootDecomposition.wl"];

FindRootPair[productTarget, Times, 3, 1,
  "TimeLimit" -> 120]
FindRootPair[sumTarget, Plus, 3, 1,
  "TimeLimit" -> 120]
```

Here 3 is the degree bound and 1 is the height bound on the primitive integer minimal polynomial of the first part. The search includes the two cubics in the question. Its returned association contains `"Parts"`, exact verification data, actual minimal-polynomial degrees, and an optimality flag. The particular order of parts, or a different but equally valid decomposition, is not part of the interface guarantee.

A faster route when one cubic is suspected is

```
RootPairFromPolynomial[productTarget, x^3+x+1, x, Times, 3]
RootPairFromPolynomial[sumTarget, x^3+x+1, x, Plus, 3]
```

These calls discover the second polynomial by factorization of a resultant. They do not assume that it is $x^3 - x + 1$.

2 What exactly is being minimized?

For $\beta \in \overline{\mathbb{Q}}$, define

$$\deg_{\mathbb{Q}}(\beta) = [\mathbb{Q}(\beta) : \mathbb{Q}].$$

This is the degree of the *minimal* polynomial over $\mathbb{Q}$. A displayed `Root` polynomial can be reducible, and a polynomial with algebraic coefficients can conceal a much larger absolute degree. Both issues are avoided by measuring degree with `MinimalPolynomial` [4].

For a nonzero algebraic number α, set

$$D_{+}(\alpha) = \min_{\alpha=\beta_1+\cdots+\beta_r} \max_i \deg_{\mathbb{Q}}(\beta_i),$$
$$D_{\times}(\alpha) = \min_{\alpha=\beta_1\cdots\beta_r} \max_i \deg_{\mathbb{Q}}(\beta_i),$$

where $r \geq 1$ is any finite integer and each β_i is algebraic. Also define $D_{\times,2}(\alpha)$ by restricting to at most two factors. The minima exist because the one-part representation is allowed and gives the upper bound $\deg_{\mathbb{Q}}(\alpha)$. Rationals, including zero, have optimal largest degree one; the zero-product case therefore requires no search.

Complex parts are allowed even when the target is real. A requirement that every part be real is a different problem; in particular, some of the field constructions below need not preserve it. Rational coefficients in an additive linear combination can be absorbed into the summands without increasing their degrees. We do not minimize the number of parts, their heights, or their typographical length after minimizing the largest degree.

A nested expression such as $\sqrt{\sqrt{2}}$ is not a representation by an ordinary quadratic `Root` over $\mathbb{Q}$: its absolute degree is four. Similarly, a mixed expression involving both sums and products is not the same optimization problem as a flat sum or a flat product.

2.1 Why factoring the input polynomial does not solve this

If the input is reduced to its minimal polynomial, that polynomial is irreducible. Nevertheless, its roots may be sums or products of lower-degree numbers. Equations (2)–(3) are examples. The relevant operation on polynomials is a *composed sum or composed product*, not ordinary factorization of the minimal polynomial itself.

The original Stack Exchange discussion includes companion-matrix and resultant constructions for an assumed factor-degree pattern [1]. They are useful starting points. The additional requirements here are exact branch selection, absolute-degree verification, clear bounds on coefficient searches, and a separate proof that no smaller maximum degree is possible.

3 Resultants: composing and recognizing roots

Suppose first that $f, g \in \mathbb{Q}[x]$ are monic of degrees m and k, with roots a_i and b_j, counted with multiplicity. The defining product identity for the resultant gives

$$C_+(x) = \operatorname{Res}_y(f(y), g(x-y)) = \prod_{i=1}^{m}\prod_{j=1}^{k}(x - a_i - b_j), \tag{5}$$

$$C_\times(x) = \operatorname{Res}_y\big(f(y), y^k g(x/y)\big) = \prod_{i=1}^{m}\prod_{j=1}^{k}(x - a_i b_j). \tag{6}$$

The second expression should be implemented as a polynomial homogenization,

$$y^k g(x/y) = \sum_{j=0}^{k} g_j x^j y^{k-j},$$

not by informal division at $y = 0$. Nonmonic input is equally valid after normalizing the resultant to monic form. The package implements these constructions using the documented `Resultant` operation [5].

For the two cubics in Section 1, exact expansion gives

$$C_\times(x) = p_\times(x), \qquad C_+(x) = p_+(x).$$

Together with the unique-real-root checks, these identities prove both requested equalities.

3.1 An annihilating polynomial is not necessarily minimal

If p is the minimal polynomial of a particular sum or product, the correct general condition is

$$p(x) \mid C_+(x) \quad \text{or} \quad p(x) \mid C_\times(x),$$

not equality. For example, composing $x^2 - 2$ with itself by addition gives

$$C_+(x) = x^2(x^2 - 8).$$

The value $2\sqrt{2}$ has minimal polynomial $x^2 - 8$, whereas the repeated value zero accounts for the extra factor. Collisions between conjugates can lower the target degree. A method that assumes $\deg p = mk$ and equates the two full polynomials will miss such decompositions.

Even a correct minimal polynomial does not determine the desired complex or real branch. Every candidate must be checked by an exact equation such as

```
RootReduce[target - beta gamma] === 0
RootReduce[target - beta - gamma] === 0
```

The package does this after considering every root of each relevant candidate polynomial. It does not infer equality from numerical proximity or from sharing a minimal polynomial.

4 Discovering the other part from one candidate polynomial

Let p be the minimal polynomial of the target α. Suppose a candidate first part β has irreducible polynomial f. Instead of introducing unknown coefficients for a second polynomial, compute

$$H_+(z) = \operatorname{Res}_y\big(f(y), p(z+y)\big), \tag{7}$$

$$H_\times(z) = \operatorname{Res}_y\big(f(y), p(zy)\big). \tag{8}$$

For products, assume $\beta \neq 0$, equivalently $f(0) \neq 0$ for an irreducible candidate other than a constant. Up to a nonzero rational scalar, the roots of these resultants are all differences $\alpha_i - \beta_j$ and all quotients α_i/β_j, respectively. Thus the minimal polynomial of the required residual part divides H.

Proposition 4.1 (Completeness for a specified first polynomial)**.** *Fix an irreducible rational polynomial f of degree at most d. Factor the appropriate H over $\mathbb{Q}$, retain irreducible factors of degree at most d, and test every pair of roots of f and those factors against the exact target. This procedure finds a two-part representation whenever one exists with first part a conjugate of a root of f and both part-degrees at most d.*

Proof. Any required residual is a root of H, so its irreducible minimal polynomial appears among the retained factors. Both its branch and the first part's branch occur in the finite root loops. Conversely, an exact successful equality is precisely a representation of the requested target. □

For both examples, taking $f(y) = y^3 + y + 1$ yields a factorization with degree/exponent profile

$$H(z) \doteq (z^3 - z + 1)^3 h_{18}(z),$$

where h_{18} is irreducible of degree eighteen and $\doteq$ means equality up to a nonzero rational constant. At bound $d = 3$, only the cubic needs branch testing. The multiplicity three is not an error; resultants remember all conjugate combinations. The independent validation script verifies both factorization profiles exactly.

4.1 A finite height search without a second height bound

Every algebraic number has a unique primitive integer minimal polynomial with positive leading coefficient. Write it as

$$f(x) = c_m x^m + \cdots + c_0, \qquad \gcd(c_0, \ldots, c_m) = 1, \quad c_m > 0,$$

and define its coefficient height by $\max_j |c_j|$. The function `FindRootPair` enumerates such polynomials with $m \leq d$ and height at most h, rejects reducible candidates, and uses (7) or (8) to recover the other part. Nonmonic candidates are included: restricting to monic integer polynomials would silently restrict the search to algebraic integers.

The second polynomial has *no coefficient-height restriction*. For a completed run, failure means only that there is no pair meeting the degree bound for which at least one part has primitive integer minimal-polynomial height at most h. The code may also return `"TimedOut"` or `"CandidateLimit"`, which make still weaker statements. Increasing h eventually discovers any existing pair, but a nonexistence proof does not follow from any one finite height cutoff.

When the target degree is $n > d$, a rational first part is impossible in a successful pair with second degree at most d. Also $n \leq md$, so the search starts at

$$m = \max\{2, \lceil n/d \rceil\}.$$

This eliminates many irrelevant candidate degrees. The coefficient enumeration is exponential in d; it is a practical method for small degrees and small heights, not a polynomial-time claim.

5 Lower bounds that certify global optimality

Theorem 5.1 (Prime-divisor obstruction)**.** *Let α lie in the field generated by finitely many algebraic numbers of degrees at most d. Every prime divisor of $[\mathbb{Q}(\alpha) : \mathbb{Q}]$ is at most d. Consequently, if $P(n)$ denotes the largest prime divisor of $n > 1$ and $P(1) = 1$, then*

$$D_+(\alpha) \geq P(\deg_{\mathbb{Q}} \alpha), \qquad D_\times(\alpha) \geq P(\deg_{\mathbb{Q}} \alpha).$$

Proof. For each generator β_i of degree $m_i \le d$, take the splitting field N_i of its minimal polynomial. Its Galois group acts faithfully on m_i roots and is therefore a subgroup of S_{m_i}. In particular, every prime dividing $[N_i : \mathbb{Q}]$ is at most m_i.

The compositum N of the N_i is finite Galois, and restriction embeds $\mathrm{Gal}(N/\mathbb{Q})$ into the product of the $\mathrm{Gal}(N_i/\mathbb{Q})$. Hence every prime divisor of $[N : \mathbb{Q}]$ is at most d. Since $\mathbb{Q}(\alpha) \subseteq N$, the tower law makes $[\mathbb{Q}(\alpha) : \mathbb{Q}]$ a divisor of $[N : \mathbb{Q}]$, proving the assertion. □

This is stronger than a restriction to sums or products: it applies to every rational expression in the proposed low-degree generators. In particular, a number of prime degree p cannot be improved below degree p by adding more parts.

Corollary 5.2 (Optimality of the question's examples)**.** *For the two targets of Section 1,*

$$\boxed{D_+(\alpha_+) = 3, \qquad D_\times(\alpha_\times) = 3.}$$

Proof. Both target degrees are nine, so Theorem 5.1 excludes largest degree one or two. Their displayed cubic decompositions attain three. □

5.1 Bounds that depend on the number of parts

If α is a sum or product of r numbers of degrees $m_1, \dots, m_r$, then

$$\deg_{\mathbb{Q}} \alpha \le [\mathbb{Q}(\beta_1, \dots, \beta_r) : \mathbb{Q}] \le \prod_{i=1}^{r} m_i \le d^r . \tag{9}$$

Thus a pair requires $d \ge \lceil \sqrt{\deg_{\mathbb{Q}} \alpha} \rceil$. This is a *two-part* bound and is not valid as a bound on arbitrary-length decompositions. The package labels a violation of $n \le d^2$ accordingly.

One must not replace the inequality in (9) with an unqualified claim that n divides $\prod_i m_i$: intermediate fields need not be normal. The prime-divisor theorem instead works in a compositum of normal closures, where the required group-order argument is valid.

The prime bound is a lower bound, not a general formula for the optimum. Degree-four numbers, for instance, need not be sums of quadratic numbers. Stronger obstructions can use the normal closure: all splitting groups of polynomials of degree at most four are solvable, so a nonsolvable normal closure rules out all such generators. For a complete additive decision, the next section uses the entire subfield lattice.

6 A complete algorithm for any finite number of summands

Let $L/\mathbb{Q}$ be a finite normal extension containing α, for example the splitting field of its minimal polynomial. Characteristic zero makes this a Galois extension. The basic field-theoretic tools are standard [10]; the reductions and algorithmic details are proved here. A closely related trace approach to low-degree additive decomposition is discussed in Altamirano's report [11].

Lemma 6.1 (Additive descent)**.** *If $\alpha \in L$ is a sum of algebraic numbers of degrees at most d, then it is a sum of elements of L, each still of degree at most d.*

Proof. Choose a finite Galois extension $M/\mathbb{Q}$ containing L and every summand. Put $G = \mathrm{Gal}(M/\mathbb{Q})$ and $H = \mathrm{Gal}(M/L)$. Since $L/\mathbb{Q}$ is normal, H is a normal subgroup of G. Define the rational-linear averaging map

$$P(u) = \frac{1}{|H|} \sum_{h \in H} h(u).$$

Then $P(M) \subseteq L$ and $P(\alpha) = \alpha$. Normality of H implies $P(gu) = gP(u)$ for every $g \in G$. Therefore every automorphism fixing a summand β also fixes $P(\beta)$. Equivalently,

$$P(\beta) \in L \cap \mathbb{Q}(\beta).$$

It follows that $\deg_{\mathbb{Q}} P(\beta) \leq \deg_{\mathbb{Q}} \beta$. Applying P to the whole sum gives the required decomposition. Zero terms may be omitted. □

For a fixed bound d, define the rational vector subspace

$$V_d(L) = \sum_{\substack{E \text{ a subfield of } L \\ [E:\mathbb{Q}] \leq d}} E \quad \subseteq L. \tag{10}$$

The sum here is a sum of vector spaces, not their compositum as fields.

Theorem 6.2 (Complete additive criterion)**.** *An algebraic number $\alpha \in L$ satisfies $D_+(\alpha) \leq d$ if and only if $\alpha \in V_d(L)$.*

Proof. If a decomposition exists, Lemma 6.1 puts each summand in L without increasing its degree. Each belongs to a subfield of degree at most d, proving membership in $V_d(L)$. Conversely, any element of the vector-space sum in (10) is a finite sum of elements belonging to those low-degree subfields. Each such element has degree at most the degree of its field. □

6.1 Turning the criterion into exact linear algebra

The Galois correspondence gives all subfields as $E = L^H$ for subgroups $H \leq \mathrm{Gal}(L/\mathbb{Q})$. Fix a power basis $1, \theta, \ldots, \theta^{N-1}$, where $N = [L : \mathbb{Q}]$. Represent each automorphism by an $N \times N$ rational matrix A_σ. A rational basis of L^H is obtained from

$$\bigcap_{\sigma \in H} \ker(A_\sigma - I). \tag{11}$$

The subfield degree is $[\mathrm{Gal}(L/\mathbb{Q}) : H]$.

For each d, collect bases of all fields of degree at most d, and solve for the target coordinates in their rational span. To construct summands of controlled degree, retain independent *original* basis vectors. Arbitrarily row-reducing and then treating the mixed vectors as low-degree elements would be wrong: a linear combination of vectors from different fields may have much larger degree.

If the retained vectors are $v_1, \ldots, v_s$ and

$$\alpha = \sum_{i=1}^{s} q_i v_i, \qquad q_i \in \mathbb{Q},$$

the output parts are $q_i v_i$ with zero terms removed. There are at most N such parts. Thus no externally imposed length bound is needed. Test d in increasing order, starting at the prime lower bound and ending at $n = \deg_{\mathbb{Q}} \alpha$. At $d = n$, the field $\mathbb{Q}(\alpha)$ itself is available, so the procedure must succeed.

```
model = BuildNormalFieldModel[sumTarget,
  "TimeLimit" -> 1800, "MaxNormalDegree" -> 48];
If[! FailureQ[model],
  CompleteSumDecomposition[sumTarget, model]
]
```

The theorem is unconditional; the implementation has explicit resource limits. It returns a failure object, not a mathematical negative answer, when normal-field construction or subgroup enumeration cannot be completed within them.

6.2 A useful obstruction example

For $\alpha = 2^{1/4}$, its normal closure is $\mathbb{Q}(2^{1/4}, i)$ of degree eight with dihedral Galois group. Its quadratic subfields are $\mathbb{Q}(i)$, $\mathbb{Q}(\sqrt{2})$, and $\mathbb{Q}(\sqrt{-2})$. Their vector-space sum is $\mathbb{Q}(i, \sqrt{2})$ and does not contain $2^{1/4}$. There are no cubic subfields, because three does not divide eight. The additive optimum is therefore four, although the prime lower bound is only two. This illustrates why a lower bound should not be mistaken for an exact general answer.

7 A complete algorithm for at most two factors

Trace is additive, not multiplicative. Applying trace independently to factors does not preserve their product. The following norm argument supplies a complete two-factor reduction, including the possibility that the factors lie outside the chosen normal field.

Lemma 7.1 (A factor and its relative norm)**.** *Let $L/\mathbb{Q}$ be finite Galois and let β be algebraic. Set*

$$E = L \cap \mathbb{Q}(\beta), \qquad k = [L(\beta) : L].$$

Then

$$[\mathbb{Q}(\beta) : \mathbb{Q}] = k[E : \mathbb{Q}], \qquad \mathrm{N}_{L(\beta)/L}(\beta) \in E. \tag{12}$$

Proof. Take a finite Galois overfield $M/\mathbb{Q}$ containing L and β. Let $H = \mathrm{Gal}(M/L)$ and $K = \mathrm{Gal}(M/\mathbb{Q}(\beta))$. Since H is normal, HK is a subgroup. Its fixed field is E, and the usual index formula gives

$$[\mathbb{Q}(\beta) : E] = [HK : K] = [H : H \cap K] = [L(\beta) : L] = k.$$

The conjugates of β over L form its H-orbit. Their product is the relative norm. This product is fixed by H, and it is also fixed by K: an element of K fixes β and permutes its H-orbit because H is normal. Therefore the norm belongs to the fixed field of HK, namely E. □

Theorem 7.2 (Finite binary-product criterion)**.** *Let $\alpha \in L^\times$. There exist algebraic β, γ with $\alpha = \beta\gamma$ and $\deg_{\mathbb{Q}} \beta, \deg_{\mathbb{Q}} \gamma \le d$ if and only if there exist an integer $1 \le k \le d$ and subfields $E, F \subseteq L$ such that*

$$k[E : \mathbb{Q}] \le d, \qquad k[F : \mathbb{Q}] \le d, \qquad \alpha^k \in E^\times F^\times. \tag{13}$$

Proof. Suppose $\alpha = \beta\gamma$. Since $\gamma = \alpha/\beta$ and $\alpha \in L$, the extensions $L(\beta)$ and $L(\gamma)$ are identical; call their common degree over L k. Put $E = L \cap \mathbb{Q}(\beta)$ and $F = L \cap \mathbb{Q}(\gamma)$. Lemma 7.1 gives the two degree inequalities. Multiplicativity of the relative norm gives

$$\alpha^k = \mathrm{N}(\beta)\,\mathrm{N}(\gamma) \in E^\times F^\times.$$

This proves necessity.

Conversely, write $\alpha^k = \delta\epsilon$ with $\delta \in E^\times$ and $\epsilon \in F^\times$. Choose any kth root β of δ, and set $\gamma = \alpha/\beta$. Then $\gamma^k = \epsilon$. Since β satisfies $X^k - \delta$ over E, and γ satisfies $X^k - \epsilon$ over F,

$$\deg_{\mathbb{Q}} \beta \le k[E : \mathbb{Q}] \le d, \qquad \deg_{\mathbb{Q}} \gamma \le k[F : \mathbb{Q}] \le d.$$

Their product is exactly α by construction. □

The exponent k records a possible extension outside L. Omitting it would impose an unnecessary restriction to factors already in L. Nor should k be confused with a bound only on relative degree: the factors' *absolute* degrees are verified after construction.

7.1 Multiplicative membership becomes a subspace intersection

The remaining condition in (13) has a particularly simple exact test:

$$\alpha^k \in E^\times F^\times \quad \Longleftrightarrow \quad E \cap \alpha^k F \neq \{0\}. \tag{14}$$

Indeed, if $\alpha^k = \delta\epsilon$, then $\delta = \alpha^k\epsilon^{-1}$ lies in the intersection. Conversely, a nonzero $\delta = \alpha^k\eta$ with $\eta \in F$ gives $\alpha^k = \delta\eta^{-1}$.

Let B_E, B_F have columns equal to rational coordinate bases of E, F, and let T_{α^k} be the matrix for multiplication by α^k in L. Then the intersection is nonzero exactly when

$$\begin{bmatrix} B_E & -T_{\alpha^k} B_F \end{bmatrix} \begin{pmatrix} u \\ v \end{pmatrix} = 0 \tag{15}$$

has a nonzero solution. Every nonzero solution gives $\delta = B_E u \neq 0$: otherwise the independence of both field bases and invertibility of multiplication by α^k would force $u = v = 0$. All of this is rational linear algebra; a unit-group computation is not needed for this two-factor criterion.

Use any such δ to form

```
beta = RootReduce[delta^(1/k)];
gamma = RootReduce[alpha/beta];
```

A principal complex root is acceptable. Defining the other factor as an exact quotient resolves the branch compatibility automatically; independently taking two principal roots would not do so.

There are finitely many subfields and finitely many $k \leq d$. Searching increasing d therefore gives the exact two-factor optimum. The first bound tested may be

$$\max\{P(n), \lceil\sqrt{n}\rceil\},$$

and the search succeeds by $d = n$. The implementation is

```
CompleteProductPairDecomposition[productTarget, model]
```

with a complete normal-field model containing the target. The returned degree minimum is stored in

`"MinimumLargestDegreeForTwoFactors"`.

This key has exactly the two-factor scope. The independent prime-bound flag can additionally prove global optimality over all numbers of factors, as it does for the question's product.

8 Constructing the normal field in Mathematica

The package builds its normal-field model using documented algebraic-number operations rather than an undocumented subfield-list function. It first obtains every root of the target minimal polynomial and calls

```
reps = ToNumberField[allRoots, All];
```

The `All` argument matters: the documentation says that it selects the smallest common field, whereas the default can select a larger extension [6]. Since all roots are included, this smallest field is the splitting field. A primitive generator θ is extracted from the common `AlgebraicNumber` representations.

For coordinates in a fixed power basis, the package combines `ToNumberField[value, theta]` with `AlgebraicNumberPolynomial`. The latter returns the representing polynomial in the generator, *not* the

minimal polynomial of the represented value [7]. A nonintegral supplied generator can be automatically replaced by an integral one, so the code uses the returned generator consistently and checks that subsequent representations keep it.

Every conjugate of θ lies in the splitting field. Replacing θ by each conjugate gives all automorphisms and their rational matrices. Their multiplication table is computed by their action on θ. A subgroup enumeration starts with the identity subgroup, adjoins group elements, and repeatedly closes under multiplication. In a finite group, a subset containing the identity and closed under multiplication is a subgroup. Repeating this process obtains the full subgroup lattice; duplicate sorted subgroups are removed.

The fixed-field bases are obtained by `NullSpace`; additive coefficients are obtained by `LinearSolve` [8]. The code checks each fixed-field dimension against its subgroup index. A supplied model is expected to be an unmodified model produced by `BuildNormalFieldModel`, not an arbitrary association with a manually asserted completeness flag.

8.1 Cost and reuse

Normal closures can be much larger than the input degree, and subgroup lattices can be large. These methods establish finite exact algorithms, not universal practicality. Their default limits are a normal degree of 48, 20,000 subgroups, and a construction time limit. Each can be changed through the corresponding options.

In the two examples, the cubics have discriminants -31 and -23. Both splitting groups are S_3; their splitting fields are disjoint over $\mathbb{Q}$, because a nontrivial common Galois quotient would force the same quadratic subfield or the same splitting field. Hence the common normal closure has group $S_3 \times S_3$ and degree 36. The degree-nine target fields coincide with $\mathbb{Q}(a, b)$, since both target degrees are nine and $[\mathbb{Q}(a, b) : \mathbb{Q}] \leq 9$. Thus a single model can be reused for both targets.

An independent finite-group enumeration gives 60 subgroups, with the following fixed-field degree distribution:

Field degree	1	2	3	4	6	9	12	18	36
Number	1	3	6	1	20	9	4	15	1

This describes the mathematical size of the example; it is not a measured Wolfram-kernel performance result. The small resultant search is the preferable first computation for these particular inputs.

9 More than two factors: a different optimization problem

A successful pair search gives an upper bound for $D_\times$, but the minimum over pairs need not be the minimum over all products. Likewise, failing to find a pair does not rule out a useful decomposition into three or more parts.

Take the same a, b as before and let c be the real root of $x^3 + x + 3$. The product abc has degree 27; this was independently checked by forming and factoring its composed-product polynomial. Its three cubic factors show $D_\times(abc) \leq 3$, while the prime-divisor bound gives equality. No product of *two* numbers of degrees at most three can equal it, because such a product has degree at most nine.

There is an analogous additive warning:

$$\sqrt{2} + \sqrt{3} + \sqrt{5}$$

has the irreducible polynomial

$$x^8 - 40x^6 + 352x^4 - 960x^2 + 576.$$

It is a sum of three quadratics, but not a sum of two quadratics by the degree bound. The complete additive algorithm handles this without fixing the number of summands.

9.1 A practical dictionary search for any bounded number of parts

The function `DictionaryRootDecomposition` accepts a finite dictionary of exact low-degree numbers. It enumerates multisets of up to $r - 1$ dictionary entries, allowing repetition, and forms the residual last part:

$$\gamma = \alpha - \sum \beta_i \quad \text{or} \quad \gamma = \frac{\alpha}{\prod \beta_i}.$$

The residual need not belong to the dictionary. Its actual minimal-polynomial degree is checked, followed by exact verification of the full identity.

```
threeSquareRoots = RootReduce[Sqrt[2]+Sqrt[3]+Sqrt[5]];
DictionaryRootDecomposition[threeSquareRoots,
  {Sqrt[2], Sqrt[3], Sqrt[5]}, Plus, 2, 3]

c = Root[3 + # + #^3 &, 1];
threeCubics = RootReduce[a b c];
DictionaryRootDecomposition[threeCubics,
  {a, b, c}, Times, 3, 3]
```

Both results, when obtained, meet the independent prime lower bound and are therefore globally optimal. A dictionary can be populated with all conjugates of low-height irreducible polynomials, selected known constants, or outputs of earlier searches. Zero entries are excluded in product search for a nonzero target.

This method is complete *within its dictionary and part-count bounds*. Its negative status does not establish global nonexistence. Exhausting successively larger polynomial-height, dictionary-size, and part-count bounds in a fair diagonal search eventually finds any existing finite decomposition. However, that is only a semidecision procedure: it does not make a failed lower-degree search terminate. Consequently it does not, by itself, compute the globally minimal degree.

9.2 The boundary of the present solution

The binary norm proof does not automatically extend to a product of arbitrarily many factors. For two factors, $L(\beta) = L(\alpha/\beta)$ provides a common relative extension degree, and multiplicative membership reduces to the intersection of two linear spaces. Neither simplification should be assumed for an arbitrary list of factors. A repeated greedy pair split also lacks a proof of global optimality.

Accordingly, this article does *not* supply a complete, terminating algorithm for $D_{\times}(\alpha)$ on every algebraic input. It supplies a complete two-factor algorithm, bounded multi-factor searches, and global certificates that settle the supplied examples and other cases attaining a proven lower bound. This limitation is explicit in the package names and result records, rather than being hidden behind a generic simplification command.

10 Coefficient solving and other useful heuristics

The alternative closest to the original discussion introduces unknown rational coefficients for f and g, builds a composed polynomial, and solves coefficient equations. If $mk = n$ and the target polynomial is monic and irreducible, equality to the target is the appropriate condition. Otherwise impose divisibility by using a polynomial remainder.

A schematic coefficient-equation construction is

```
equations = Thread[CoefficientList[
  Expand[composedPolynomial - targetPolynomial], x] == 0];
solutions = Solve[equations, parameters];
```

This is not a universal decomposition algorithm. First, `Solve` does not automatically restrict the unknown coefficients to rational numbers: algebraic coefficient solutions can conceal large absolute degrees. Second, coefficient systems can have parameters and difficult rational-point conditions. Third, fixing a product factor's constant coefficient to one is not generally without loss of generality over $\mathbb{Q}$: the required scaling might require an algebraic root and increase the absolute degree.

A constant-coefficient or depressed-cubic ansatz can be excellent for recovering a particular example, provided its scope is stated. It should not serve as a global impossibility test. Similarly, a numerical relation or `RootApproximant`-style guess can suggest candidates, but only an exact check and a degree lower bound turn that suggestion into a certified optimal answer.

The residual-resultant method has a useful asymmetry: it enumerates rational candidate coefficients for just one part and discovers the other polynomial exactly. This avoids solving a large nonlinear coefficient system and avoids bounding both polynomial heights.

11 Package interface, safeguards, and validation

The source is included in Appendix A and as a separate `RootDecomposition.wl` file. Its main interfaces are summarized below.

Function	Guarantee or purpose
`RootDegree`	Actual degree over $\mathbb{Q}$; rejects inexact real input.
`DegreePrimeLowerBound`	A bound valid for every finite number of parts.
`VerifyRootDecomposition`	Exact residual, degree list, and prime-bound optimality flag.
`RootPairFromPolynomial`	Complete branch search for a specified first polynomial.
`FindRootPair`	Finite first-polynomial height search; no second height bound.
`DictionaryRootDecomposition`	Bounded-length multiset search with an unrestricted residual last part.
`BuildNormalFieldModel`	Complete normal-field and subfield model, or explicit resource failure.
`CompleteSumDecomposition`	Global additive optimum, with no part-count or coefficient bound, given a complete model.
`CompleteProductPairDecomposition`	Exact optimum for at most two factors in arbitrary algebraic fields.

The result record keeps the parts as a list and additionally supplies an inactive sum or product. This prevents a convenient display from immediately erasing the decomposition by normalization. The inactive expression can be activated when needed. Every proposed algebraic identity is checked with `RootReduce`, and every part-degree is independently computed with `MinimalPolynomial`.

Search success uses tagged `Catch`/`Throw` to leave nested loops. This is deliberate: `Return` inside a `Do` can exit the loop rather than the entire surrounding function, as the official documentation illustrates [9]. Resource exhaustion is kept distinct from a completed negative bounded search.

11.1 What was tested, and what was not

The included `validate_independently.py` was executed with SymPy 1.14.0. Its 32 checks passed. They include the exact composed-polynomial identities, irreducibility and real-root counts for both examples, residual-resultant factor profiles, nonmonic and zero-root cases, the degree-27 three-cubic example, the degree-eight three-square-root example, a full subgroup enumeration of $S_3 \times S_3$, and small rational-matrix checks of the additive and binary-product criteria. Numerical values are included only as supplementary orientation. The generated results are in `validation.json` and `validation.txt`.

The script also checks balanced source delimiters in the Wolfram files. This is only a basic static check, not a Wolfram parser or an execution test. The Wolfram evaluation connector returned an HTTP 404 error, and no local Wolfram kernel was available. Therefore **the supplied Wolfram package and its test suite have not been executed in a Wolfram kernel here**. No timing, test-pass count, or runtime compatibility claim for that package is implied by the independent mathematical checks.

The `tests.wlt` file supplies regression tests for the requested identities, branch verification, searches, exact-input rejection, nonmonic polynomials, zero roots, and small complete-field examples. In Mathematica, run

```
TestReport["tests.wlt"]
```

from the extracted directory, or provide its absolute path. The last tests construct normal fields and are more expensive than the direct resultant checks. A finite resource limit may need to be raised for a particular machine or kernel version.

12 Recommended workflow

Start with the exact minimal polynomial and the prime-divisor lower bound. For a small target, run the residual-resultant search at small degree and height. A found decomposition whose largest degree equals the lower bound solves the original global optimization immediately; this is all that is needed for the two examples.

When additive optimality remains unresolved, build a complete normal-field model and test subfield spans. When an unrestricted two-factor product optimum is needed, use the norm-and-intersection algorithm on that model. For arbitrary-length products, use a bounded dictionary or polynomial-generated dictionary and keep its search limits visible. A successful exact identity is always useful, but call it globally optimal only when a valid lower bound or a complete algorithm for the relevant scope justifies that claim.

The essential separation is therefore

candidate discovery + exact verification + scope-correct optimality proof.

For the numbers in the question, all three stages are completed and the answer is exactly the two cubic roots originally proposed.

References

[1] Vladimir Reshetnikov, *Factor a polynomial root into roots of smallest possible degree*, Mathematica Stack Exchange, question 105933 and answers by yarchik and Daniel Lichtblau. https://mathematica.stackexchange.com/questions/105933/. Accessed September 6, 2026.

[2] Wolfram Research, *Root*, Wolfram Language documentation. https://reference.wolfram.com/language/ref/Root.html.

[3] Wolfram Research, *RootReduce*, Wolfram Language documentation. https://reference.wolfram.com/language/ref/RootReduce.html.

[4] Wolfram Research, *MinimalPolynomial*, Wolfram Language documentation. https://reference.wolfram.com/language/ref/MinimalPolynomial.html.

[5] Wolfram Research, *Resultant*, Wolfram Language documentation. https://reference.wolfram.com/language/ref/Resultant.html.

[6] Wolfram Research, *ToNumberField*, Wolfram Language documentation. https://reference.wolfram.com/language/ref/ToNumberField.html.

[7] Wolfram Research, *AlgebraicNumberPolynomial*, Wolfram Language documentation. https://reference.wolfram.com/language/ref/AlgebraicNumberPolynomial.html.

[8] Wolfram Research, *NullSpace* and *LinearSolve*, Wolfram Language documentation.
https://reference.wolfram.com/language/ref/NullSpace.html;
https://reference.wolfram.com/language/ref/LinearSolve.html.

[9] Wolfram Research, *Return*, especially Scope examples, Wolfram Language documentation. https://reference.wolfram.com/language/ref/Return.html.

[10] J. S. Milne, *Fields and Galois Theory*, version 5.10, September 2022. https://www.jmilne.org/math/CourseNotes/FT.pdf.

[11] Christian Altamirano, *A Problem in Algebraic Number Theory*, MIT UROP+ report, August 31, 2017; mentor Atticus Christensen. https://math.mit.edu/research/undergraduate/urop-plus/documents/2017/Altamirano.pdf.

A Complete Wolfram Language package

This is the same source as the separate `RootDecomposition.wl` file. The mathematical guarantees are conditional on successful exact execution; the execution limitation in Section 11 applies to this listing. The package uses no undocumented Mathematica functions. Set variables appearing in user-supplied polynomial arguments to be unassigned before calling the polynomial interfaces.

```
(* ::Package:: *)
(* Exact low-degree algebraic decomposition.
   See article.pdf for proofs, guarantees, and the limits of each search.
   This source uses only documented Wolfram Language functionality.
   The accompanying .wlt tests are intended for an actual Wolfram kernel.
*)

BeginPackage["RootDecomposition`"];
RootDegree::usage = "RootDegree[a] gives the degree of an exact algebraic number over Q.";
DegreePrimeLowerBound::usage = "DegreePrimeLowerBound[a] is a lower bound on the largest degree in ANY finite sum or product representing a.";
SumComposedPolynomial::usage = "SumComposedPolynomial[f,g,x] is the monic polynomial of all pairwise sums of roots of f and g.";
ProductComposedPolynomial::usage = "ProductComposedPolynomial[f,g,x] is the monic polynomial of all pairwise products of roots of f and g.";
VerifyRootDecomposition::usage = "VerifyRootDecomposition[a,parts,Plus|Times] verifies the identity exactly and reports the actual degrees.";
RootPairFromPolynomial::usage = "RootPairFromPolynomial[a,f,x,Plus|Times,d] searches all conjugates of f and recovers a second factor/summand of degree at most d.";
FindRootPair::usage = "FindRootPair[a,Plus|Times,d,h] searches primitive integer first polynomials of degree at most d and coefficient height at most h. The other polynomial has no height bound.";
DictionaryRootDecomposition::usage = "DictionaryRootDecomposition[a,atoms,Plus|Times,d,r] searches up to r parts, all but the last drawn with repetition from atoms. The last part may be any algebraic number of degree at most d.";
BuildNormalFieldModel::usage = "BuildNormalFieldModel[a] constructs a normal field containing a, its automorphisms, and the complete subfield lattice. This can be very expensive.";
CompleteSumDecomposition::usage = "CompleteSumDecomposition[a,model] minimizes the largest degree over ALL finite sums. Omit model to construct it. Resource failures are not negative answers.";
CompleteProductPairDecomposition::usage = "CompleteProductPairDecomposition[a,model] minimizes the largest degree over products of AT MOST TWO factors, allowing factors outside the normal field. It does not optimize arbitrary-length products.";

Begin["`Private`"];
qQ[t_] := MatchQ[t, _Integer | _Rational];
qPolyQ[p_, x_] := PolynomialQ[p, x] && AllTrue[CoefficientList[p, x], qQ];
fail[tag_, text_] := Failure[tag, <|"MessageTemplate" -> text|>];
rr[a_] := Quiet[Check[RootReduce[a], $Failed]];
eqQ[a_, b_] := SameQ[rr[a - b], 0];
nonzeroVectorQ[v_List] := AnyTrue[v, ! SameQ[#, 0] &];
monic[p_, x_] := Expand[p/Coefficient[p, x, Exponent[p, x]]];

RootDegree[a_] := Module[{x, p},
  If[! FreeQ[a, _Real], Return[fail["InexactInput", "Use exact algebraic input, not floating-point numbers."]]];
  p = Quiet[Check[MinimalPolynomial[a, x], $Failed]];
  If[p === $Failed || ! qPolyQ[p, x] || Exponent[p, x] < 1,
    Return[fail["NotExactAlgebraic", "Could not obtain a rational minimal polynomial."]]];
  Exponent[p, x]
];

DegreePrimeLowerBound[a_] := Module[{n = RootDegree[a]},
  If[FailureQ[n], Return[n]];
  If[n == 1, 1, Max[First /@ FactorInteger[n]]]
];

rootAt[p_, x_, j_Integer] := With[
  {fun = Function @@ {p /. x -> Slot[1]}, index = j}, Root[fun, index]
];

SumComposedPolynomial[f_, g_, x_Symbol] := Module[{y},
  If[! qPolyQ[f, x] || ! qPolyQ[g, x] || Min[Exponent[f, x], Exponent[g, x]] < 1,
    Return[fail["PolynomialInput", "Supply nonconstant rational polynomials in an unassigned variable."]]];
```

```
  monic[Resultant[f /. x -> y, g /. x -> x - y, y], x]
];

ProductComposedPolynomial[f_, g_, x_Symbol] := Module[{y, n, transformed},
  If[! qPolyQ[f, x] || ! qPolyQ[g, x] || Min[Exponent[f, x], Exponent[g, x]] < 1,
    Return[fail["PolynomialInput", "Supply nonconstant rational polynomials in an unassigned variable."
     ]]];
  n = Exponent[g, x];
  (* Homogenize explicitly: no spurious denominator or zero-root loss. *)
  transformed = Sum[Coefficient[g, x, j] x^j y^(n - j), {j, 0, n}];
  monic[Resultant[f /. x -> y, transformed, y], x]
];

VerifyRootDecomposition[a_, parts_List, op : (Plus | Times)] := Module[
  {degrees, n, value, residual, lower, largest},
  If[Length[parts] == 0, Return[fail["EmptyParts", "Supply at least one part."]]];
  n = RootDegree[a]; If[FailureQ[n], Return[n]];
  degrees = RootDegree /@ parts;
  If[AnyTrue[degrees, FailureQ], Return[fail["InvalidPart", "Every part must be exact and algebraic."
     ]]];
  value = Apply[op, parts]; residual = rr[a - value];
  If[residual === $Failed, Return[fail["VerificationFailure", "Exact verification did not complete."
     ]]];
  lower = If[n == 1, 1, Max[First /@ FactorInteger[n]]];
  largest = Max[degrees];
  <|"Verified" -> SameQ[residual, 0], "Residual" -> residual,
    "InputDegree" -> n, "Degrees" -> degrees, "LargestDegree" -> largest,
    "GlobalLowerBound" -> lower,
    "GloballyOptimalByPrimeBound" -> (SameQ[residual, 0] && largest == lower)|>
];

certificate[a_, parts_List, op_, scope_String] := Module[{v},
  v = VerifyRootDecomposition[a, parts, op];
  If[FailureQ[v] || ! TrueQ[v["Verified"]],
    Return[fail["CertificateFailure", "The proposed identity was not certified."]]];
  Join[<|"Status" -> If[TrueQ[v["GloballyOptimalByPrimeBound"]],
        "CertifiedGloballyOptimal", "CertifiedDecomposition"],
      "Parts" -> parts, "Operation" -> op,
      "Expression" -> Apply[Inactive[op], parts], "SearchScope" -> scope|>, v]
];

pairFromPolynomial[a_, p_, f_, x_, op_, d_] := Module[
  {y, z, h, gs, beta, gamma, c, tag}, Catch[
  h = Switch[op,
    Plus, Resultant[f /. x -> y, p /. x -> z + y, y],
    Times, Resultant[f /. x -> y, p /. x -> z y, y]];
  If[! qPolyQ[h, z] || SameQ[h, 0],
    Throw[fail["ResultantFailure", "Residual resultant computation failed."], tag]];
  gs = Select[First /@ FactorList[h], 1 <= Exponent[#, z] <= d &];
  Do[
    beta = rootAt[f, x, i];
    If[op === Times && eqQ[beta, 0], Continue[]];
    Do[
      gamma = rootAt[g, z, j];
      If[eqQ[a, op[beta, gamma]],
        c = certificate[a, {beta, gamma}, op, "Two parts; specified first polynomial"];
        Throw[Join[c, <|"FirstPolynomial" -> f,
          "SecondPolynomial" -> (monic[g, z] /. z -> x)|>], tag]],
      {g, gs}, {j, 1, Exponent[g, z]}],
    {i, 1, Exponent[f, x]}];
  <|"Status" -> "NoPairFromCandidate", "FirstPolynomial" -> f|>
, tag]];

RootPairFromPolynomial[a_, f_, x_Symbol, op : (Plus | Times), d_Integer?Positive] := Module[{p, n},
  n = RootDegree[a]; If[FailureQ[n], Return[n]];
  If[! qPolyQ[f, x] || ! TrueQ[IrreduciblePolynomialQ[f]] ||
      ! (1 <= Exponent[f, x] <= d),
    Return[fail["CandidateInput", "The candidate must be irreducible over Q, with degree from 1 through
      d."]]];
  If[op === Times && SameQ[f /. x -> 0, 0],
    Return[fail["ZeroCandidate", "The multiplicative candidate cannot have zero as a root."]]];
  p = MinimalPolynomial[a, x];
  pairFromPolynomial[a, p, f, x, op, d]
```

```
];

Options[FindRootPair] = {"TimeLimit" -> 60, "MaxCandidates" -> Infinity};
FindRootPair[a_, op : (Plus | Times), d_Integer?Positive, h_Integer?Positive,
    OptionsPattern[]] := Module[
  {x, p, n, tag, tried = 0, coeffs, f, answer, lowerDegree, cap, seconds},
  n = RootDegree[a]; If[FailureQ[n], Return[n]];
  If[n <= d, Return[certificate[a, {rr[a]}, op, "Trivial one-part representation"]]];
  If[n > d^2, Return[<|"Status" -> "NoTwoFactorDecomposition",
      "Reason" -> "Input degree exceeds d^2", "InputDegree" -> n,
      "DegreeBound" -> d, "Scope" -> "At most two parts only"|>]];
  p = MinimalPolynomial[a, x]; lowerDegree = Max[2, Ceiling[n/d]];
  cap = OptionValue["MaxCandidates"]; seconds = OptionValue["TimeLimit"];
  TimeConstrained[
    Catch[
      Do[
        coeffs = Join[IntegerDigits[index, 2 height + 1, m] - height, {lead}];
        If[Max[Abs[coeffs]] != height || Apply[GCD, coeffs] != 1, Continue[]];
        f = Expand[coeffs . x^Range[0, m]];
        If[op === Times && First[coeffs] == 0, Continue[]];
        If[! TrueQ[IrreduciblePolynomialQ[f]], Continue[]];
        If[tried >= cap, Throw[<|"Status" -> "CandidateLimit", "CandidatesTested" -> tried|>, tag]];
        tried++;
        answer = pairFromPolynomial[a, p, f, x, op, d];
        If[FailureQ[answer], Throw[answer, tag]];
        If[KeyExistsQ[answer, "Parts"],
          Throw[Join[answer, <|"CandidatesTested" -> tried,
            "FirstPolynomialHeightBound" -> h, "DegreeBound" -> d|>], tag]],
        {height, 1, h}, {m, lowerDegree, d}, {lead, 1, height},
        {index, 0, (2 height + 1)^m - 1}];
      <|"Status" -> "NotFoundWithinBounds", "CandidatesTested" -> tried,
        "DegreeBound" -> d, "FirstPolynomialHeightBound" -> h,
        "Scope" -> "At most two parts; at least one primitive integer minimal polynomial has height <=
     h"|>,
      tag],
    seconds,
    <|"Status" -> "TimedOut", "CandidatesTested" -> tried,
      "DegreeBound" -> d, "FirstPolynomialHeightBound" -> h|>]
];

Options[DictionaryRootDecomposition] = {"TimeLimit" -> 60};
DictionaryRootDecomposition[a_, atoms_List, op : (Plus | Times),
    d_Integer?Positive, maxParts_Integer?Positive, OptionsPattern[]] := Module[
  {n, ds, pool, tag, indices, chosen, last, lastDegree, tried = 0},
  n = RootDegree[a]; If[FailureQ[n], Return[n]];
  ds = RootDegree /@ atoms;
  If[AnyTrue[ds, FailureQ] || AnyTrue[ds, # > d &],
    Return[fail["DictionaryInput", "All dictionary entries must be exact algebraic numbers of degree <=
      d."]]];
  pool = DeleteDuplicates[rr /@ atoms];
  If[op === Times, pool = Select[pool, ! eqQ[#, 0] &]];
  If[n <= d, Return[certificate[a, {rr[a]}, op, "One part"]]];
  If[pool === {}, Return[<|"Status" -> "NotFoundWithinDictionary"|>]];
  TimeConstrained[
    Catch[
      Do[
        indices = If[Length[pool] == 1, ConstantArray[1, r - 1],
          1 + IntegerDigits[index, Length[pool], r - 1]];
        If[! OrderedQ[indices], Continue[]];
        chosen = pool[[indices]]; tried++;
        last = rr[If[op === Plus, a - Total[chosen], a/Apply[Times, chosen]]];
        If[last === $Failed, Throw[fail["ArithmeticFailure", "Residual reduction failed."], tag]];
        lastDegree = RootDegree[last];
        If[FailureQ[lastDegree], Throw[lastDegree, tag]];
        If[lastDegree <= d,
          Throw[certificate[a, Append[chosen, last], op,
            "Bounded dictionary, all but the last part"], tag]],
        {r, 2, maxParts}, {index, 0, Length[pool]^(r - 1) - 1}];
      <|"Status" -> "NotFoundWithinDictionary", "CombinationsTested" -> tried,
        "MaxParts" -> maxParts, "DegreeBound" -> d|>, tag],
    OptionValue["TimeLimit"], <|"Status" -> "TimedOut", "CombinationsTested" -> tried|>]
];

```

```
(* Power-basis coordinates. The model generator is an algebraic integer;
   otherwise ToNumberField can silently change the generator. *)
fieldCoordinates[a_, theta_, p_, z_, n_] := Module[{u, h},
  u = Quiet[Check[ToNumberField[a, theta], $Failed]];
  If[u === $Failed || ! FreeQ[u, ToNumberField], Return[$Failed]];
  If[Head[u] === AlgebraicNumber && ! eqQ[u[[1]], theta], Return[$Failed]];
  h = Quiet[Check[AlgebraicNumberPolynomial[u, z], $Failed]];
  If[h === $Failed || ! qPolyQ[h, z], Return[$Failed]];
  PadRight[CoefficientList[PolynomialRemainder[h, p, z], z], n]
];

fieldMulMatrix[v_List, p_, z_, n_] := Transpose[
  Table[PadRight[CoefficientList[
    PolynomialRemainder[(v . z^Range[0, n - 1]) z^j, p, z], z], n],
    {j, 0, n - 1}]
];

allSubgroups[table_List, id_Integer, limit_] := Module[
  {groups = {{id}}, pos = 1, h, k, closure, order = Length[table], tag}, Catch[
  closure[seed_] := FixedPoint[Union[#, Flatten[table[[#, #]]]] &, Union[seed, {id}]];
  While[pos <= Length[groups],
    h = groups[[pos]];
    Do[
      k = closure[Append[h, g]];
      If[! MemberQ[groups, k],
        If[Length[groups] >= limit, Throw[$Failed, tag]];
        AppendTo[groups, k]],
      {g, Complement[Range[order], h]}];
    pos++];
  groups
, tag]];

Options[BuildNormalFieldModel] = {
  "TimeLimit" -> 300, "MaxNormalDegree" -> 48, "MaxSubgroups" -> 20000};
BuildNormalFieldModel[a_, OptionsPattern[]] := Module[
  {x, z, p0, n0, roots, reps, generators, theta, p, n, images,
   autos, id, table, groups, fields, coords, imagePoly, loc},
  n0 = RootDegree[a]; If[FailureQ[n0], Return[n0]];
  TimeConstrained[
    If[n0 == 1,
      Return[<|"Generator" -> 1, "Polynomial" -> z - 1,
        "Variable" -> z, "Dimension" -> 1,
        "Automorphisms" -> {{{1}}}, "MultiplicationTable" -> {{1}},
        "Subfields" -> {<|"Degree" -> 1, "Basis" -> {{1}}, "Subgroup" -> {1}|>},
        "CompleteSubfieldLattice" -> True|>]];
    p0 = MinimalPolynomial[a, x];
    roots = Table[rootAt[p0, x, j], {j, n0}];
    reps = Quiet[Check[ToNumberField[roots, All], $Failed]];
    If[reps === $Failed, Return[fail["PrimitiveElementFailure", "Could not construct the splitting
     field."]]];
    generators = Cases[reps, AlgebraicNumber[t_, _List] :> t, Infinity];
    If[generators === {}, Return[fail["PrimitiveElementFailure", "No primitive-element representation
     was returned."]]];
    theta = First[generators]; p = MinimalPolynomial[theta, z]; n = Exponent[p, z];
    If[n > OptionValue["MaxNormalDegree"],
      Return[fail["NormalDegreeLimit", "Normal-field degree exceeds the configured limit; no negative
     conclusion follows."]]];
    coords[t_] := fieldCoordinates[t, theta, p, z, n];
    images = Table[coords[rootAt[p, z, j]], {j, n}];
    If[MemberQ[images, $Failed], Return[fail["NonNormalModel", "Not all conjugates could be expressed
     in the chosen field."]]];
    autos = Table[
      imagePoly = images[[i]] . z^Range[0, n - 1];
      Transpose[Table[PadRight[CoefficientList[
        PolynomialRemainder[imagePoly^j, p, z], z], n], {j, 0, n - 1}]],
      {i, n}];
    loc = FirstPosition[images, coords[theta]];
    If[MissingQ[loc], Return[fail["AutomorphismFailure", "Could not identify the identity automorphism.
     "]]];
    id = First[loc];
    table = Table[
      loc = FirstPosition[images, autos[[i]] . images[[j]]];
      If[MissingQ[loc], $Failed, First[loc]], {i, n}, {j, n}];
```

```
    If[! FreeQ[table, $Failed], Return[fail["AutomorphismFailure", "The automorphism table did not close."]]];
    groups = allSubgroups[table, id, OptionValue["MaxSubgroups"]];
    If[groups === $Failed, Return[fail["SubgroupLimit", "Subgroup enumeration exceeded its limit; no negative conclusion follows."]]];
    fields = Table[
      <|"Degree" -> n/Length[h], "Subgroup" -> h,
        "Basis" -> NullSpace[Join @@ ((# - IdentityMatrix[n] &) /@ autos[[h]])]|>,
      {h, groups}];
    If[AnyTrue[fields, Length[#["Basis"]] != #["Degree"] &],
      Return[fail["FixedFieldFailure", "Fixed-field dimensions failed the index check."]]];
    fields = SortBy[fields, #["Degree"] &];
    <|"Generator" -> theta, "Polynomial" -> p, "Variable" -> z,
      "Dimension" -> n, "Automorphisms" -> autos,
      "MultiplicationTable" -> table, "Subfields" -> fields,
      "CompleteSubfieldLattice" -> True|>,
    OptionValue["TimeLimit"], fail["TimedOut", "Normal-field construction timed out; no negative conclusion follows."]]
];

modelCoordinates[a_, model_Association] := fieldCoordinates[a,
  model["Generator"], model["Polynomial"], model["Variable"], model["Dimension"]];
modelValue[v_List, model_Association] := rr[v . model["Generator"]^Range[0, model["Dimension"] - 1]];
validModelQ[m_] := AssociationQ[m] && TrueQ[Lookup[m, "CompleteSubfieldLattice", False]];

CompleteSumDecomposition[a_, supplied_ : Automatic] := Module[
  {n, model, target, fields, candidates, basis, matrix, sol, parts, result, lower, tag}, Catch[
  n = RootDegree[a]; If[FailureQ[n], Throw[n, tag]];
  If[n == 1, Throw[Join[certificate[a, {rr[a]}, Plus, "All finite sums"],
    <|"MinimumLargestDegree" -> 1, "OptimalityScope" -> "All finite sums"|>], tag]];
  model = If[supplied === Automatic, BuildNormalFieldModel[a], supplied];
  If[FailureQ[model], Throw[model, tag]];
  If[! validModelQ[model], Throw[fail["ModelInput", "Supply an unmodified complete model from BuildNormalFieldModel."], tag]];
  target = modelCoordinates[a, model];
  If[target === $Failed, Throw[fail["FieldMembership", "The target is not in the model field."], tag]];
  lower = DegreePrimeLowerBound[a];
  Do[
    fields = Select[model["Subfields"], #["Degree"] <= d &];
    candidates = Join @@ (#["Basis"] & /@ fields);
    (* Retain original basis vectors, not mixed RowReduce vectors. *)
    basis = {};
    Do[If[MatrixRank[Append[basis, v]] > Length[basis], AppendTo[basis, v]], {v, candidates}];
    matrix = Transpose[basis];
    If[MatrixRank[Append[basis, target]] == Length[basis],
      sol = Quiet[Check[LinearSolve[matrix, target], $Failed]];
      If[sol === $Failed || ! SameQ[matrix . sol, target],
        Throw[fail["LinearSolveFailure", "The exact additive linear system failed verification."], tag]];
      parts = DeleteCases[MapThread[rr[#1 modelValue[#2, model]] &, {sol, basis}], 0];
      result = certificate[a, parts, Plus, "All finite sums, with no coefficient or length bound"];
      If[FailureQ[result], Throw[result, tag]];
      If[result["LargestDegree"] > d,
        Throw[fail["DegreeVerification", "The additive parts violated the degree bound."], tag]];
      Throw[Join[result, <|"Status" -> "CertifiedGloballyOptimal",
        "OptimalityScope" -> "All finite sums", "MinimumLargestDegree" -> d|>], tag]],
    {d, lower, n}];
  fail["InternalFailure", "The search should have succeeded by the input degree."]
, tag]];

CompleteProductPairDecomposition[a_, supplied_ : Automatic] := Module[
  {n, model, fields, power, powerVector, mult, b1, b2, matrix, kernel,
   vector, delta, beta, gamma, result, lower, tag}, Catch[
  n = RootDegree[a]; If[FailureQ[n], Throw[n, tag]];
  If[n == 1, Throw[Join[certificate[a, {rr[a]}, Times, "At most two factors"],
    <|"MinimumLargestDegreeForTwoFactors" -> 1,
      "OptimalityScope" -> "At most two factors"|>], tag]];
  model = If[supplied === Automatic, BuildNormalFieldModel[a], supplied];
  If[FailureQ[model], Throw[model, tag]];
  If[! validModelQ[model], Throw[fail["ModelInput", "Supply an unmodified complete model from BuildNormalFieldModel."], tag]];
  If[modelCoordinates[a, model] === $Failed,
    Throw[fail["FieldMembership", "The target is not in the model field."], tag]];
```

```
  lower = Max[DegreePrimeLowerBound[a], Ceiling[Sqrt[n]]];
  Do[
    Do[
      fields = Select[model["Subfields"], k #["Degree"] <= d &];
      power = rr[a^k]; powerVector = modelCoordinates[power, model];
      If[powerVector === $Failed, Throw[fail["FieldArithmetic", "Could not compute a target power in the model."], tag]];
      mult = fieldMulMatrix[powerVector, model["Polynomial"], model["Variable"], model["Dimension"]];
      Do[
        b1 = fields[[i]]["Basis"]; b2 = fields[[j]]["Basis"];
        matrix = Join[Transpose[b1], -mult . Transpose[b2], 2];
        kernel = NullSpace[matrix];
        If[kernel =!= {},
          vector = Take[First[kernel], Length[b1]] . b1;
          If[! nonzeroVectorQ[vector], Throw[fail["IntersectionFailure", "Unexpected zero vector in a nonzero intersection."], tag]];
          delta = modelValue[vector, model];
          beta = rr[delta^(1/k)]; gamma = rr[a/beta];
          result = certificate[a, {beta, gamma}, Times, "At most two factors, unrestricted algebraic fields"];
          If[FailureQ[result], Throw[result, tag]];
          If[result["LargestDegree"] > d,
            Throw[fail["DegreeVerification", "The constructed factors violated the proven degree bound."], tag]];
          Throw[Join[result, <|"OptimalityScope" -> "At most two factors",
            "MinimumLargestDegreeForTwoFactors" -> d,
            "NormExponent" -> k,
            "SubfieldDegrees" -> {fields[[i]]["Degree"], fields[[j]]["Degree"]}|>], tag]],
        {i, Length[fields]}, {j, i, Length[fields]}],
      {k, 1, d}],
    {d, lower, n}];
  fail["InternalFailure", "The two-factor search should succeed by the input degree."]
, tag]];

End[];
EndPackage[];
```

B Archive contents and rebuilding

The archive contains `article.tex`, the compiled `article.pdf`, the package, `examples.wl`, `tests.wlt`, the independent Python validator, its JSON and text results, a README, and a Makefile. No external bibliography database is required. The package listing is read from the adjacent source file during compilation.

Rebuild the PDF with two passes of pdfLaTeX:

```
pdflatex -interaction=nonstopmode -halt-on-error article.tex
pdflatex -interaction=nonstopmode -halt-on-error article.tex
```

The Python validator may be rerun independently; it requires SymPy. It neither calls Mathematica nor substitutes for the Wolfram regression tests.